Chapter 7

Advanced SQL

Database Systems:
Design, Implementation, and Management, Sixth Edition, Rob and Coronel

UNION Query

- Combine two companies customer lists where they have names in common so that the final list has no duplicate names
  - Column names and their attributes must be the same

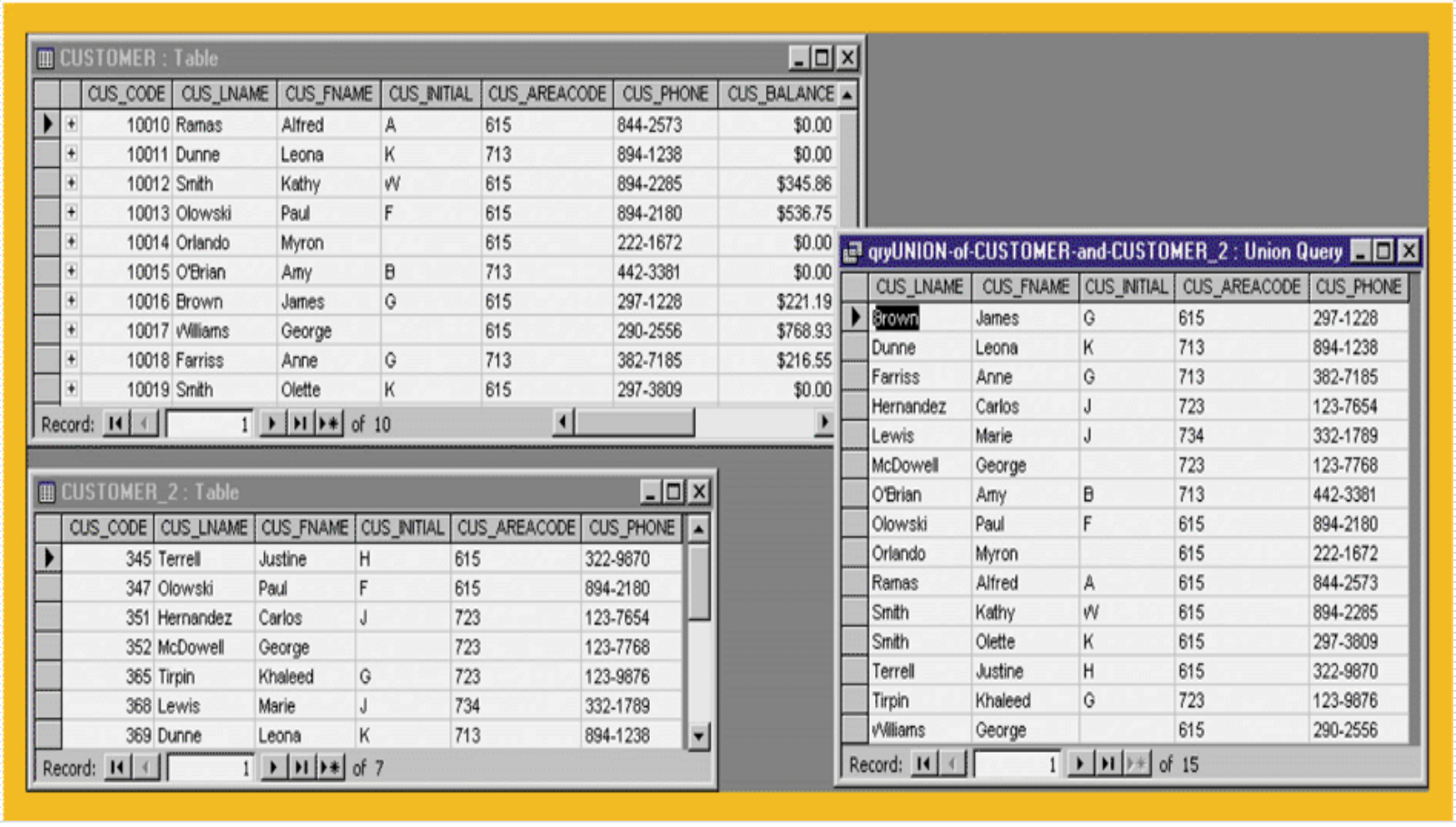
```
SELECT <COLUMN NAMES> FROM CUSTOMER

UNION

SELECT <COLUMN NAMES> FROM CUSTOMER_2;
```

- Can include more than two tables where applicable

```
SELECT <COLUMN NAMES> FROM T1 UNION
SELECT <COLUMN NAMES> FROM T2 UNION
SELECT <COLUMN NAMES> FROM T3;
```

UNION Query Result

FIGURE 7.1 UNION QUERY RESULT



CUSTOMER : Table

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_BALANCE
10010	Ramas	Alfred	A	615	844-2573	$0.00
10011	Dunne	Leona	K	713	894-1238	$0.00
10012	Smith	Kathy	W	615	894-2285	$345.86
10013	Olowski	Paul	F	615	894-2180	$536.75
10014	Orlando	Myron		615	222-1672	$0.00
10015	O'Brian	Amy	B	713	442-3381	$0.00
10016	Brown	James	G	615	297-1228	$221.19
10017	Williams	George		615	290-2556	$768.93
10018	Farriss	Anne	G	713	382-7185	$216.55
10019	Smith	Olette	K	615	297-3809	$0.00

Record: 1 of 10

CUSTOMER_2 : Table

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE
345	Terrell	Justine	H	615	322-9870
347	Olowski	Paul	F	615	894-2180
351	Hernandez	Carlos	J	723	123-7654
352	McDowell	George		723	123-7768
365	Tirpin	Khaleed	G	723	123-9876
368	Lewis	Marie	J	734	332-1789
369	Dunne	Leona	K	713	894-1238

Record: 1 of 7

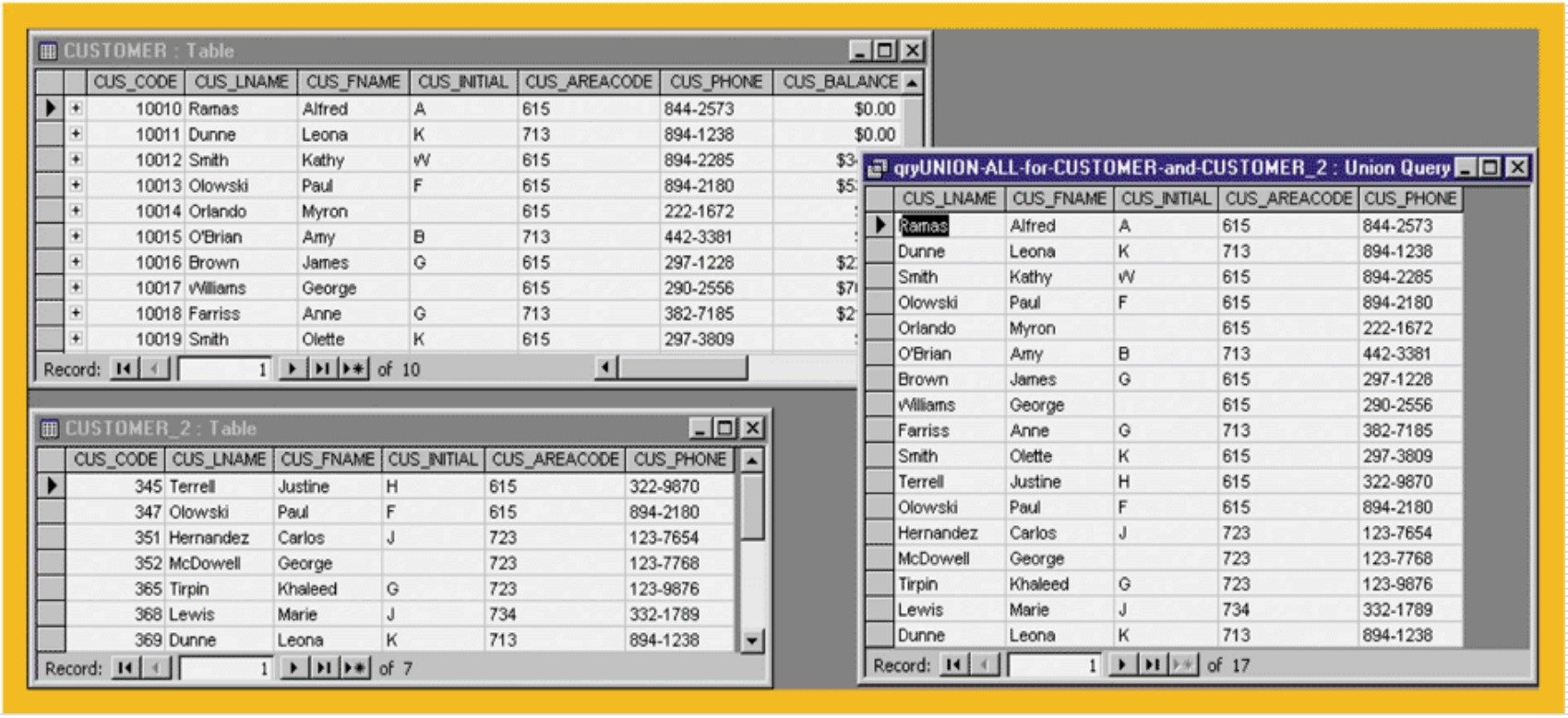
qryUNION-of-CUSTOMER-and-CUSTOMER_2 : Union Query

CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE
Brown	James	G	615	297-1228
Dunne	Leona	K	713	894-1238
Farriss	Anne	G	713	382-7185
Hernandez	Carlos	J	723	123-7654
Lewis	Marie	J	734	332-1789
McDowell	George		723	123-7768
O'Brian	Amy	B	713	442-3381
Olowski	Paul	F	615	894-2180
Orlando	Myron		615	222-1672
Ramas	Alfred	A	615	844-2573
Smith	Kathy	W	615	894-2285
Smith	Olette	K	615	297-3809
Terrell	Justine	H	615	322-9870
Tirpin	Khaleed	G	723	123-9876
Williams	George		615	290-2556

Record: 1 of 15

UNION ALL Query Result

FIGURE 7.2 UNION ALL QUERY RESULT



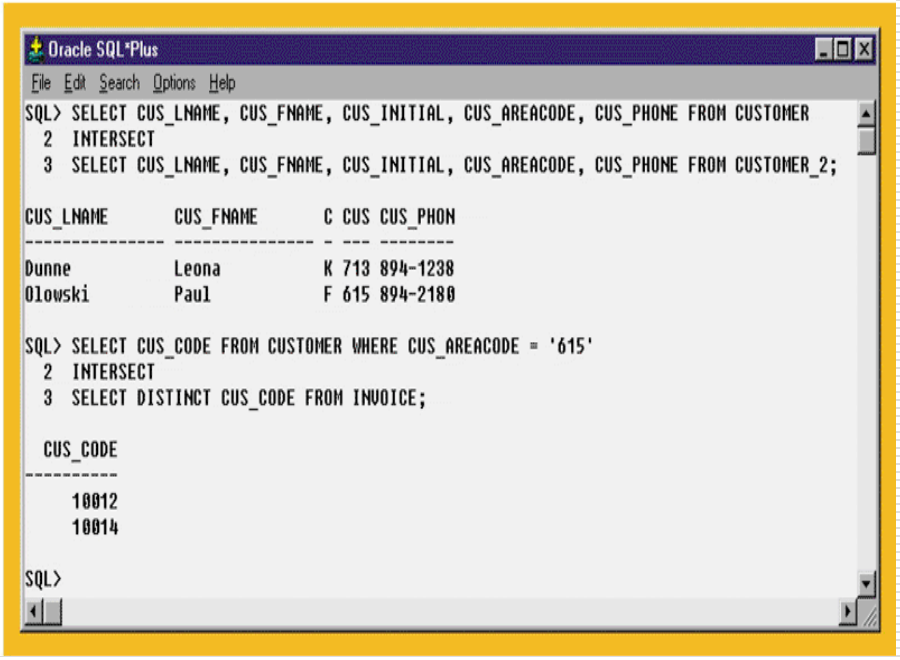
UNION ALL merges the tables but keeps duplicates in the result table

Intersect Query

- Find the rows that are in both tables
- Find rows based on another select
- Not available in MS Access
- Alternative

```
SELECT CUS_CODE FROM
CUSTOMER
WHERE CUS_AREACODE='615'
AND CUS_CODE IN
(SELECT DISTINCT
CUS_CODE FROM INVOICE);
```

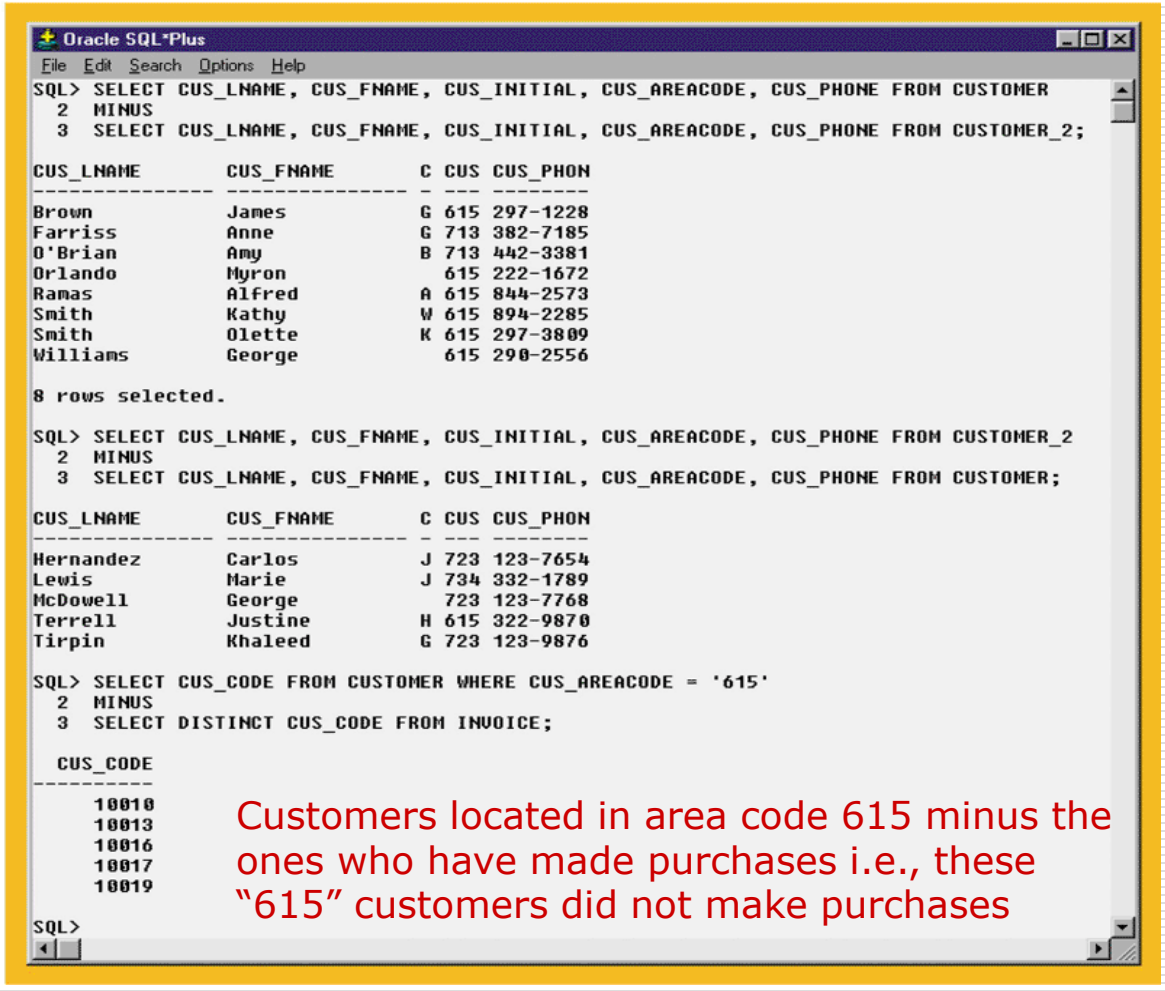
FIGURE 7.3 INTERSECT QUERY RESULT



Minus Query

- Combines rows from two queries and returns only those rows that appear in the first set but not the second
- Alternative – use **NOT IN**

FIGURE 7.4 MINUS QUERY RESULTS



SQL Join Expression Styles

TABLE 7.1 SQL JOIN EXPRESSION STYLES

JOIN CLASSIFICATION	JOIN TYPE	SQL SYNTAX EXAMPLE	DESCRIPTION
Cross	CROSS JOIN	SELECT * FROM T1, T2	Returns the Cartesian product of T1 and T2—old style
		SELECT * FROM T1 CROSS JOIN T2	Returns the Cartesian product of T1 and T2
Inner	Old-Style JOIN	SELECT * FROM T1, T2 WHERE T1.C1=T2.C1	Returns only the rows that meet the join condition in the WHERE clause—old style. Only rows with matching values are selected.
	NATURAL JOIN	SELECT * FROM T1 NATURAL JOIN T2	Returns only the rows with matching values in the matching columns. The matching columns must have the same names and similar data types.
	JOIN USING	SELECT * FROM T1 JOIN T2 USING (C1)	Returns only the rows with matching values in the columns indicated in the USING clause
	JOIN ON	SELECT * FROM T1 JOIN T2 ON T1.C1=T2.C1	Returns only the rows that meet the join condition indicated in the ON clause
Outer	LEFT JOIN	SELECT * FROM T1 LEFT OUTER JOIN T2 ON T1.C1=T2.C1	Returns rows with matching values and includes all rows from the left table (T1) with unmatched values
	RIGHT JOIN	SELECT * FROM T1 RIGHT OUTER JOIN T2 ON T1.C1=T2.C1	Returns rows with matching values and includes all rows from the right table (T2) with unmatched values
	FULL JOIN	SELECT * FROM T1 FULL OUTER JOIN T2 ON T1.C1=T2.C1	Returns rows with matching values and includes all rows from both tables (T1 and T2) with unmatched values

Additional clauses with JOIN

- JOIN USING – return only the rows with matching vales in the column indicated in the USING clause
  - This column must exist in both tables
  - SELECT column-list FROM table1 JOIN table2 USING (common-column)
- JOIN ON – when the tables have no common attribute name but the columns have the same attribute type
  - SELECT column-list FROM table1 JOIN table2 ON join-condition

SELECT Subquery Examples

TABLE 7.2 SELECT SUBQUERY EXAMPLES

SELECT SUBQUERY EXAMPLES	EXPLANATION
INSERT INTO PRODUCT SELECT * FROM P;	Inserts all rows from the table P into the PRODUCT Table. Both tables must have the same attributes. The subquery returns all rows from table P.
UPDATE PRODUCT SET P_PRICE = (SELECT AVG(P_PRICE) FROM PRODUCT) WHERE V_CODE IN (SELECT V_CODE FROM VENDOR WHERE V_AREACODE = '615');	Updates the product price to the average product price, but only for the products that are provided by vendors who have an area code equal to 615. The first subquery returns the average price; the second subquery returns the list of vendors with an area code equal to 615.
DELETE FROM PRODUCT WHERE V_CODE IN (SELECT V_CODE FROM VENDOR WHERE V_AREACODE = '615');	Deletes the PRODUCT table rows that are provided by vendors with an area code equal to '615'. The subquery returns the list of vendors' codes with area code equal to 615.

WHERE Subquery Examples

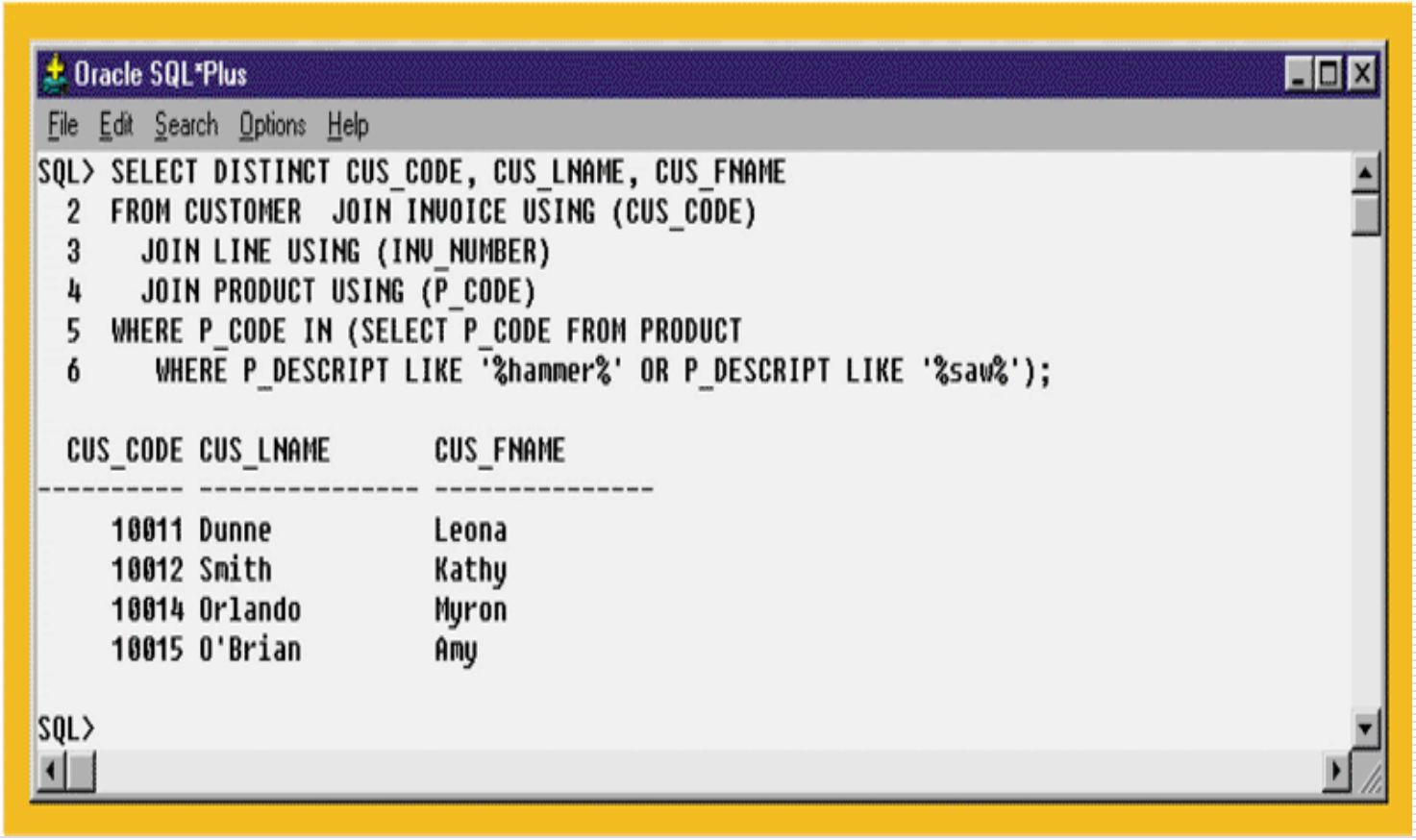
- Subquery can return
  - One single value (one column, one row)
  - A list of values (one column, multiple rows)
  - A virtual table (multicolumn, multirow set of values)
  - NULL

Other types of subqueries

- IN subqueries – you want to find all customers who have purchased something from a list of products, not just one product
- HAVING subqueries – restrict a GROUP BY by applying a conditional criteria to the grouped rows
  - List al products with the total quantity sold greater than the average quantity sold
- INLINE subqueries – can appear as a value in a the columnlist of a SELECT
  - Must return one value
    - Compute difference between product's individual price and its average price
- CORRELATED subqueries – subquery executes once for each row in the outer row (like nested loops)

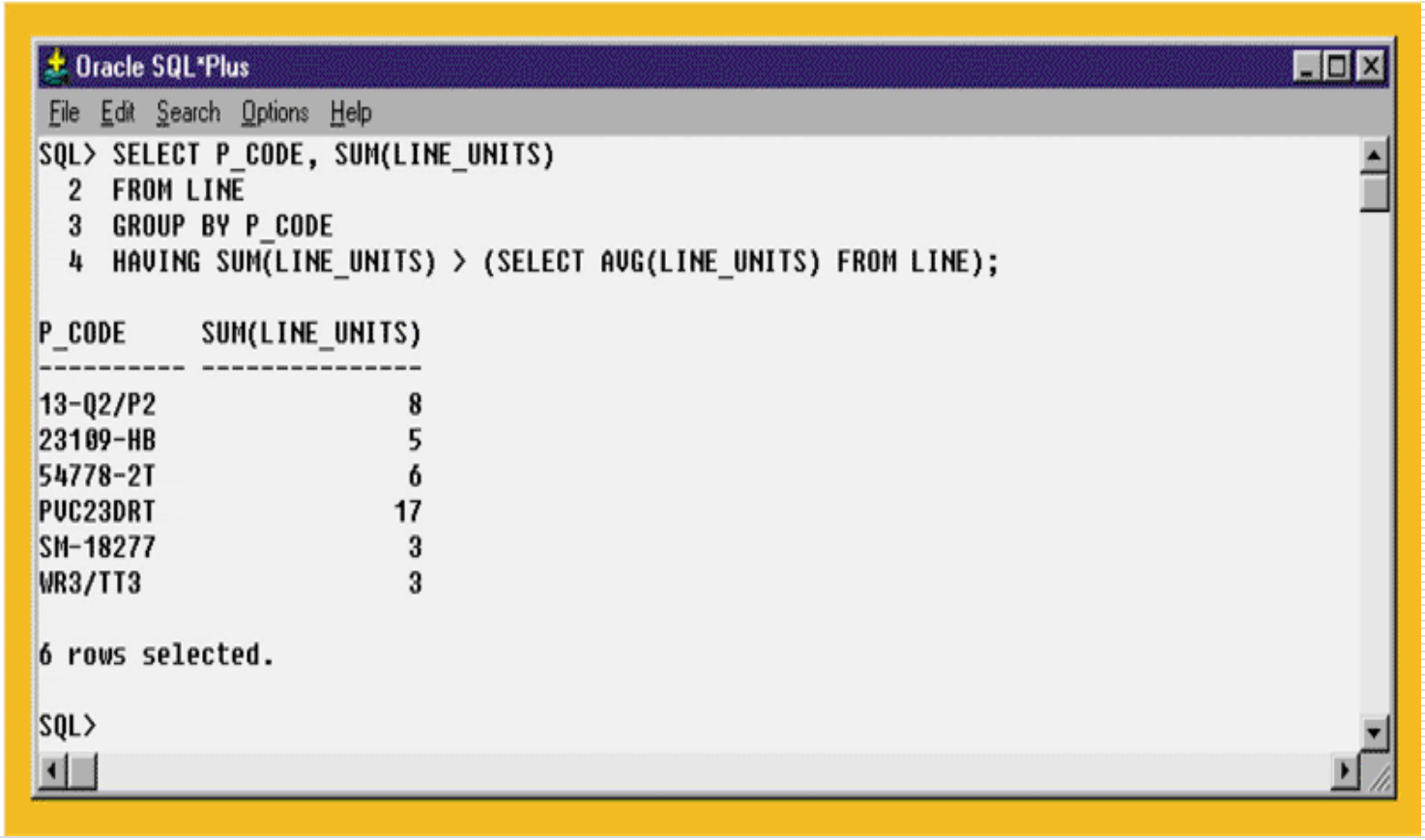
IN Subquery Example

FIGURE 7.14 IN SUBQUERY EXAMPLE



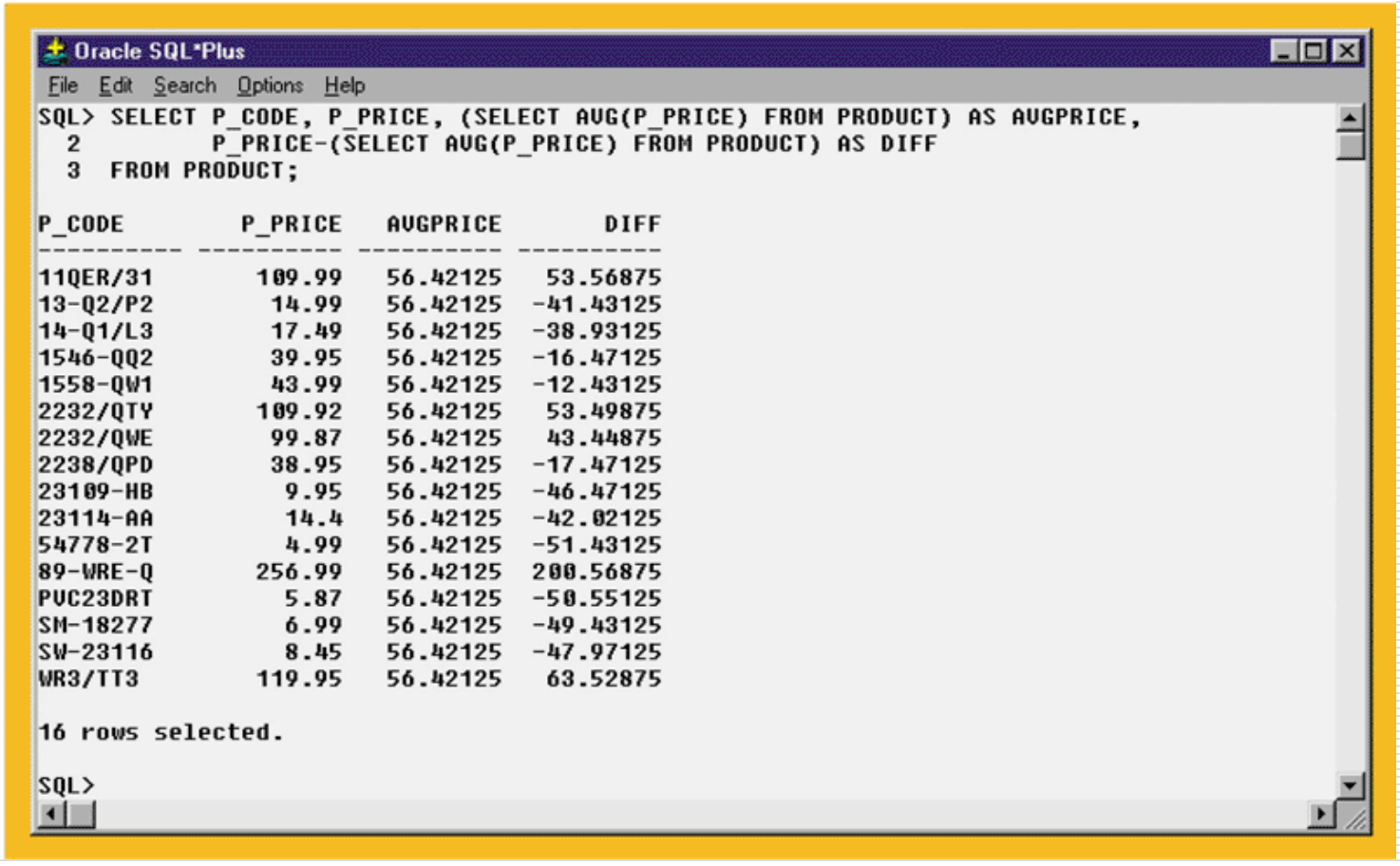
HAVING Subquery Example

FIGURE 7.15 HAVING SUBQUERY EXAMPLE



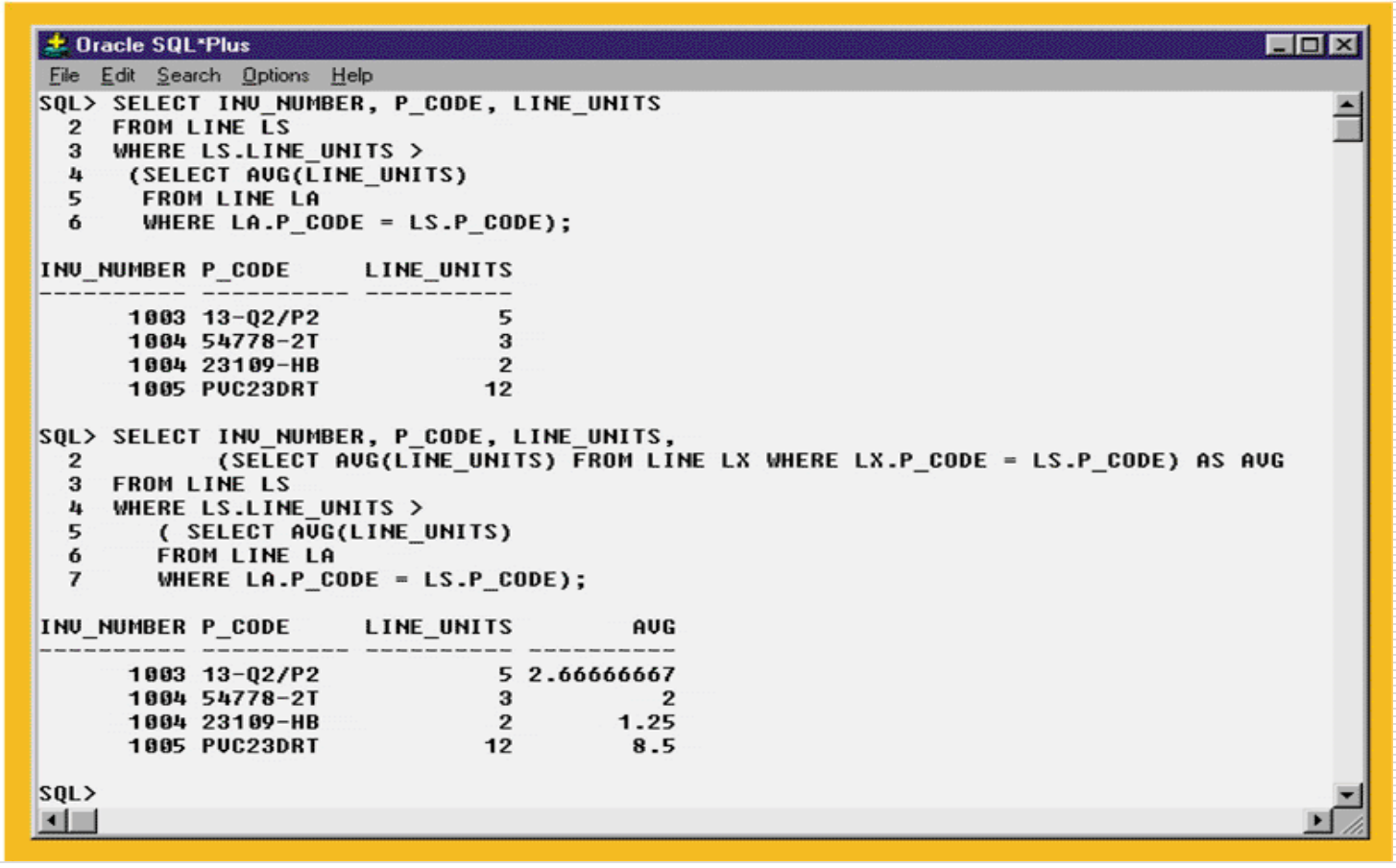
Inline Subquery Example

FIGURE 7.18 INLINE SUBQUERY EXAMPLE



Correlated Subquery Examples

FIGURE 7.20 CORRELATED SUBQUERY EXAMPLES

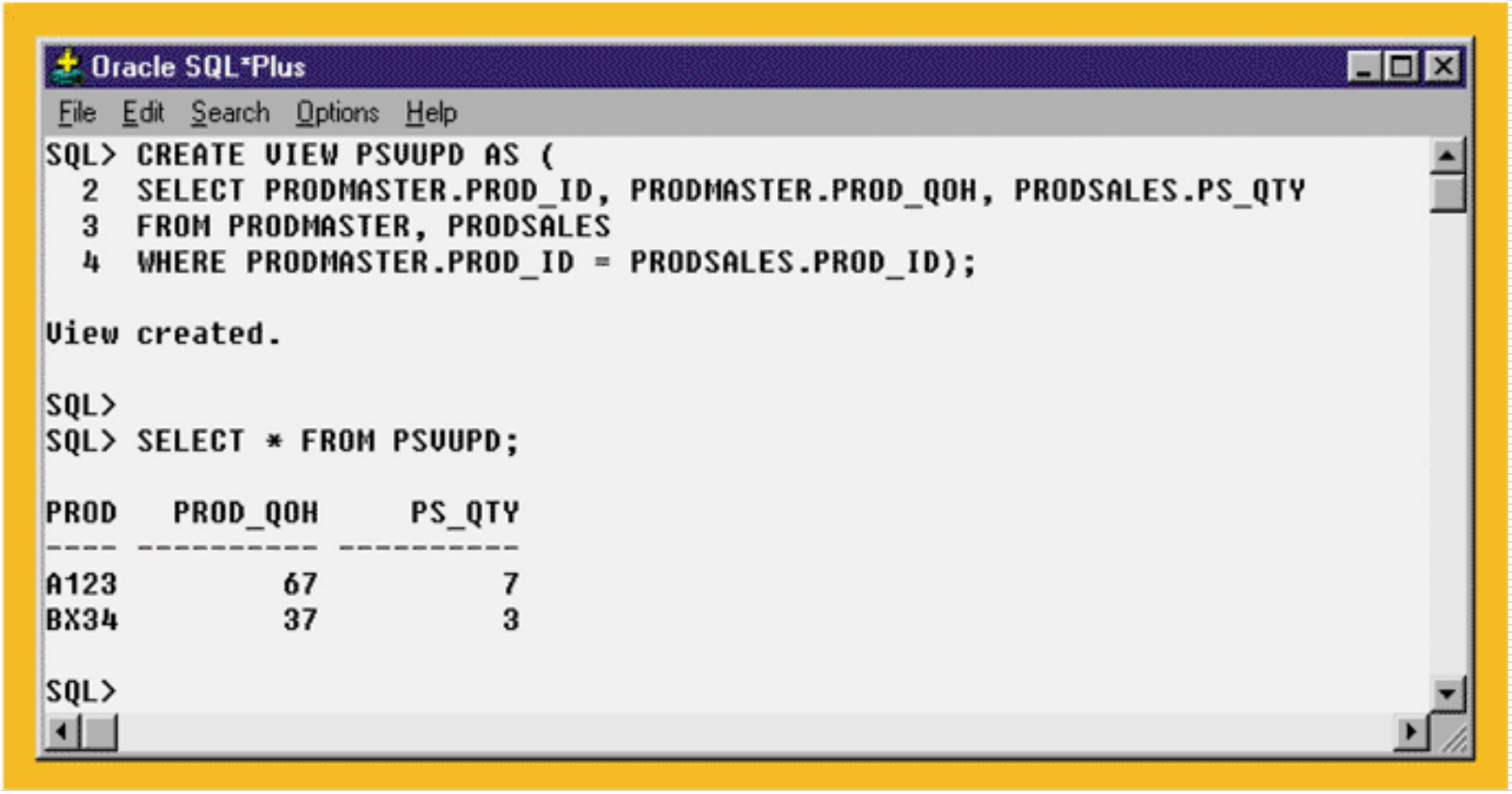


Updatable Views

- Views can be made to serve common data management tasks
  - Batch update routine – pools multiple transactions into a single batch to update a master table file in a single operation
  - Updates attributes in the base tables that are used in the view
    - Update a product's available inventory based on summary sales transactions

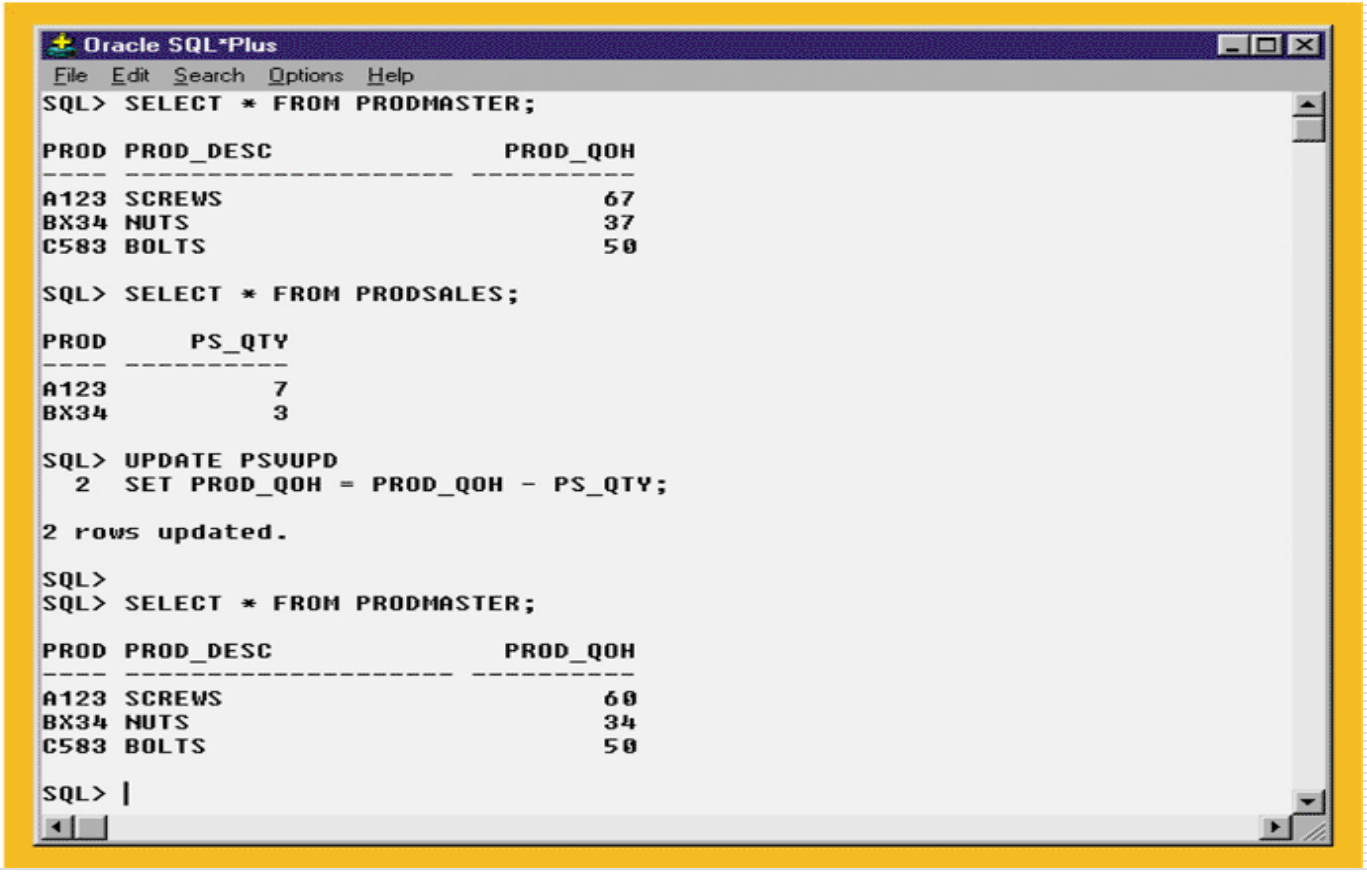
Creating an Updatable View in Oracle

FIGURE 7.26 CREATING AN UPDATABLE VIEW IN ORACLE



PRODMASTER Table Update, Using an Updatable View

FIGURE 7.27 PRODMASTER TABLE UPDATE, USING AN UPDATABLE VIEW



Procedural SQL

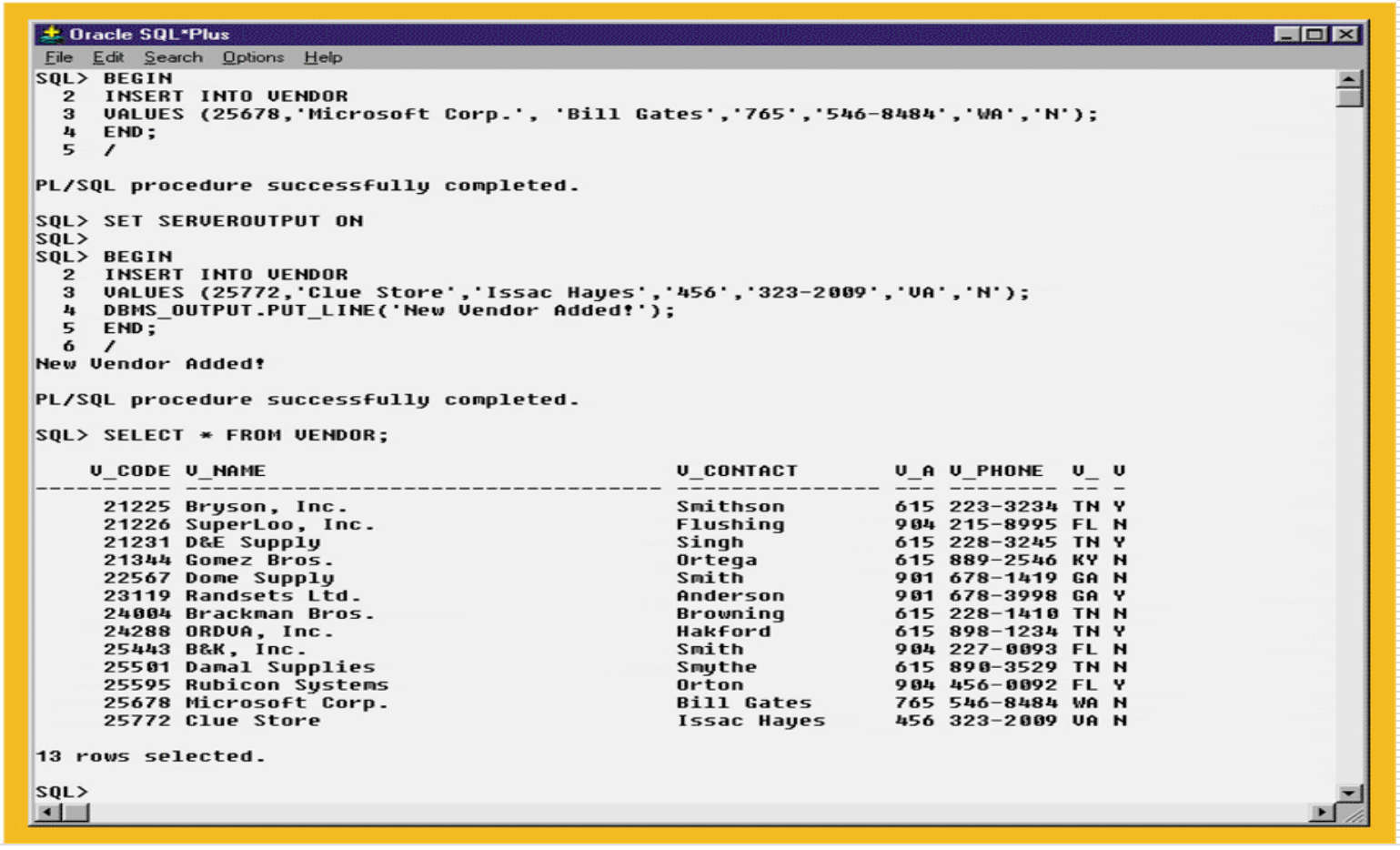
- SQL does not support the conditional execution of procedures (IF-THEN-ELSE) or loops
- One way to do this is by writing procedural code in a programming language and including SQL statements in the progam
  - This spreads out SQL code in many programs and make changes difficult
- A better way is to isolate critical code and have all applications call this shared code
  - Needed by distributed and OO databases
  - SQL 99 defined the use of **persistent stored modules** (PSMs)

Procedural SQL

- A PSM is a block of code that is stored and executed at the DBMS server
  - Represents business logic that can be shared among multiple database users
  - Access can be controlled by the DBA
  - Oracle supports this through Procedural SQL – PL/SQL

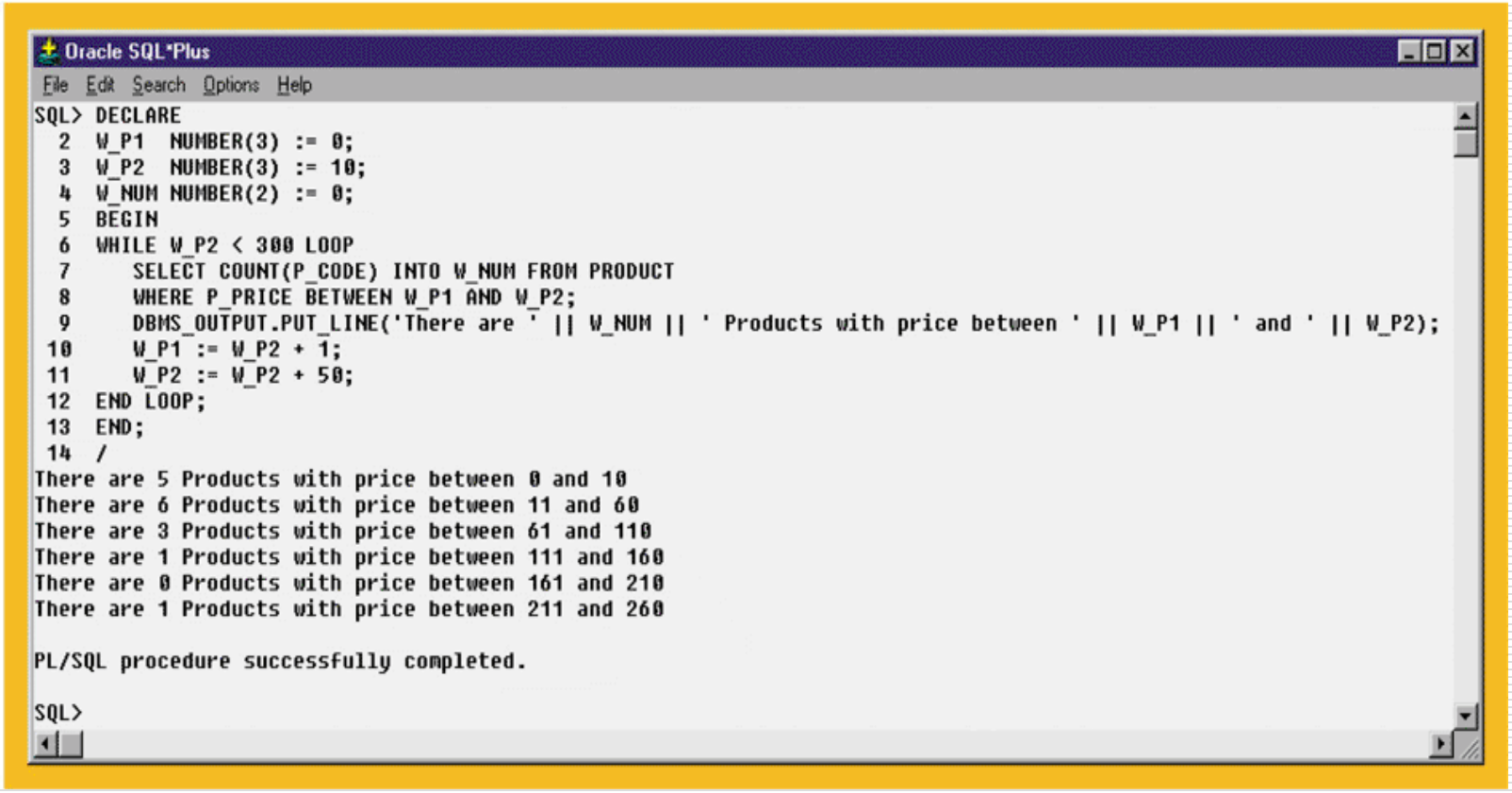
Anonymous PL/SQL Block Examples

FIGURE 7.28 ANONYMOUS PL/SQL BLOCK EXAMPLES



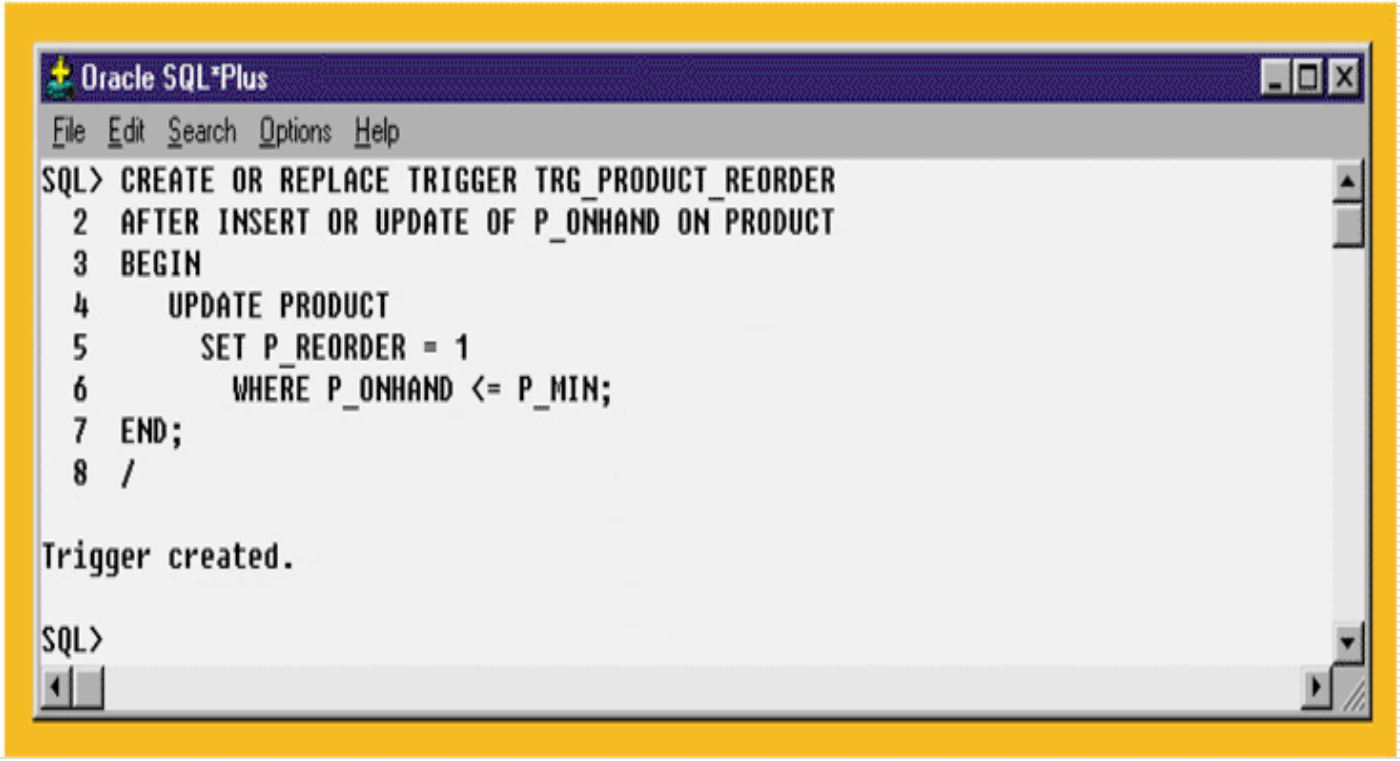
Anonymous PL/SQL Block with Variables and Loops

FIGURE 7.29 ANONYMOUS PL/SQL BLOCK WITH VARIABLES AND LOOPS



Creating a Trigger using PL/SQL

FIGURE 7.31 CREATING THE TRG_PRODUCT_REORDER TRIGGER

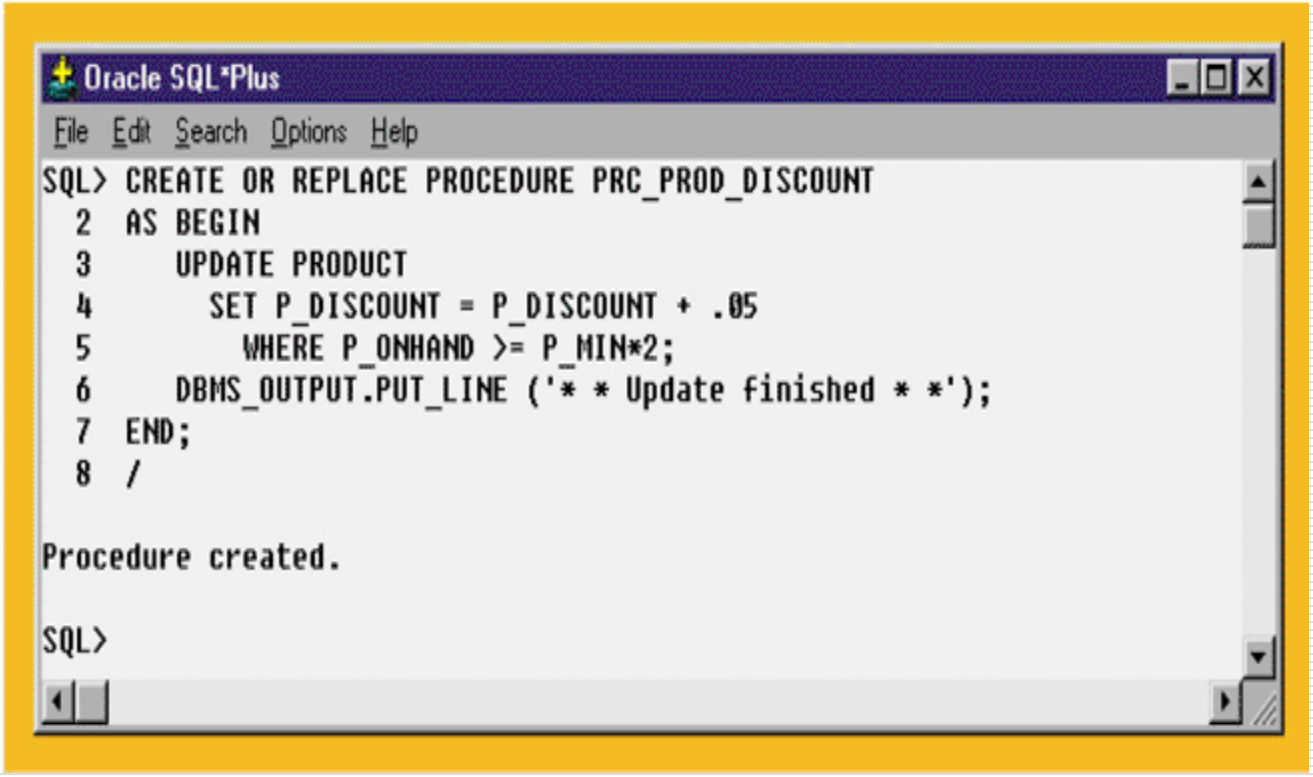


Stored Procedures: Advantages

- Substantially reduce network traffic and increase performance
- No transmission of individual SQL statements over network
- Help reduce code duplication by means of code isolation and code sharing
- Minimize chance of errors and cost of application development and maintenance

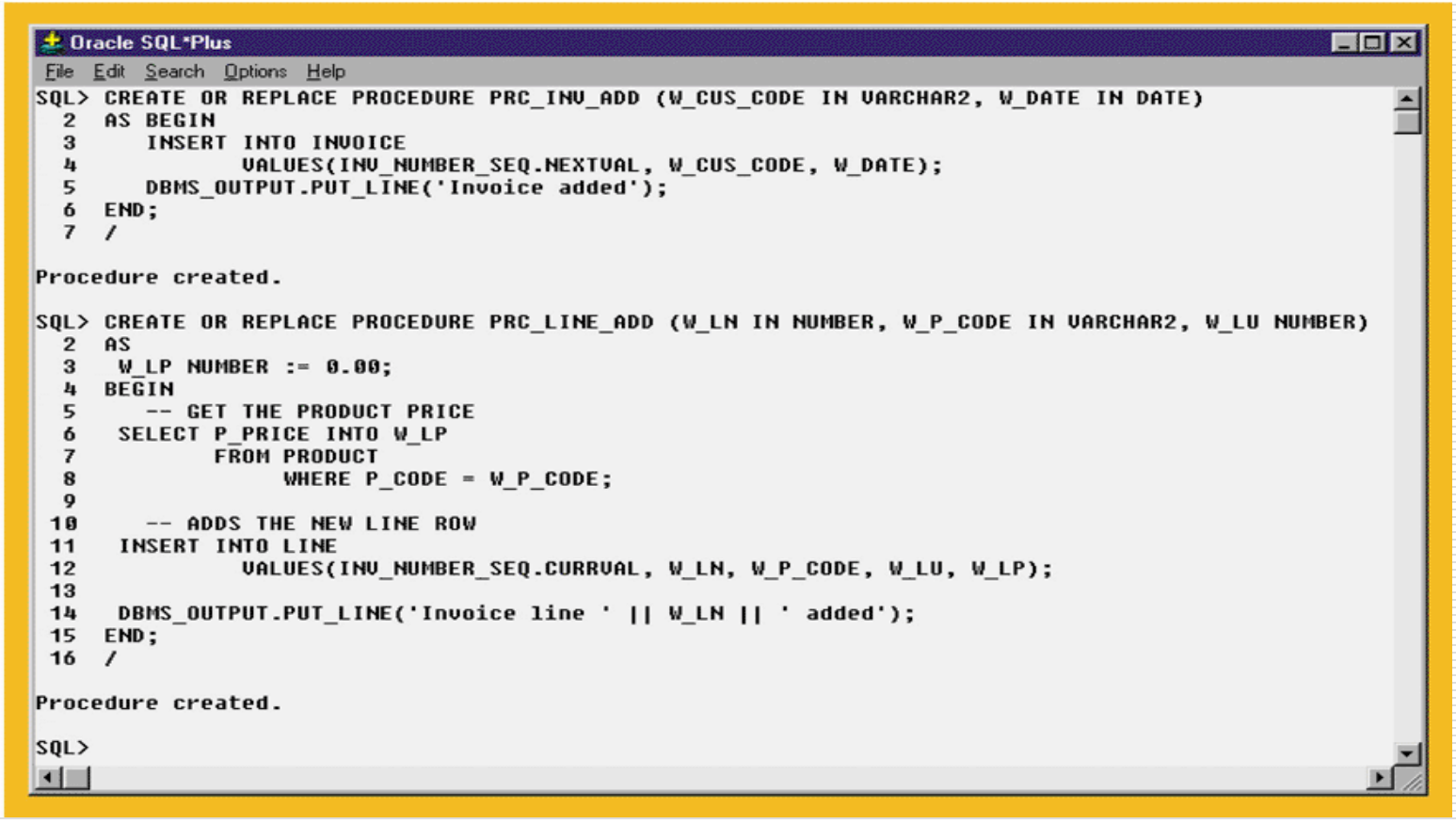
Creating the PRC_PROD_DISCOUNT Stored Procedure

FIGURE 7.41 CREATING THE PRC_PROD_DISCOUNT STORED PROCEDURE



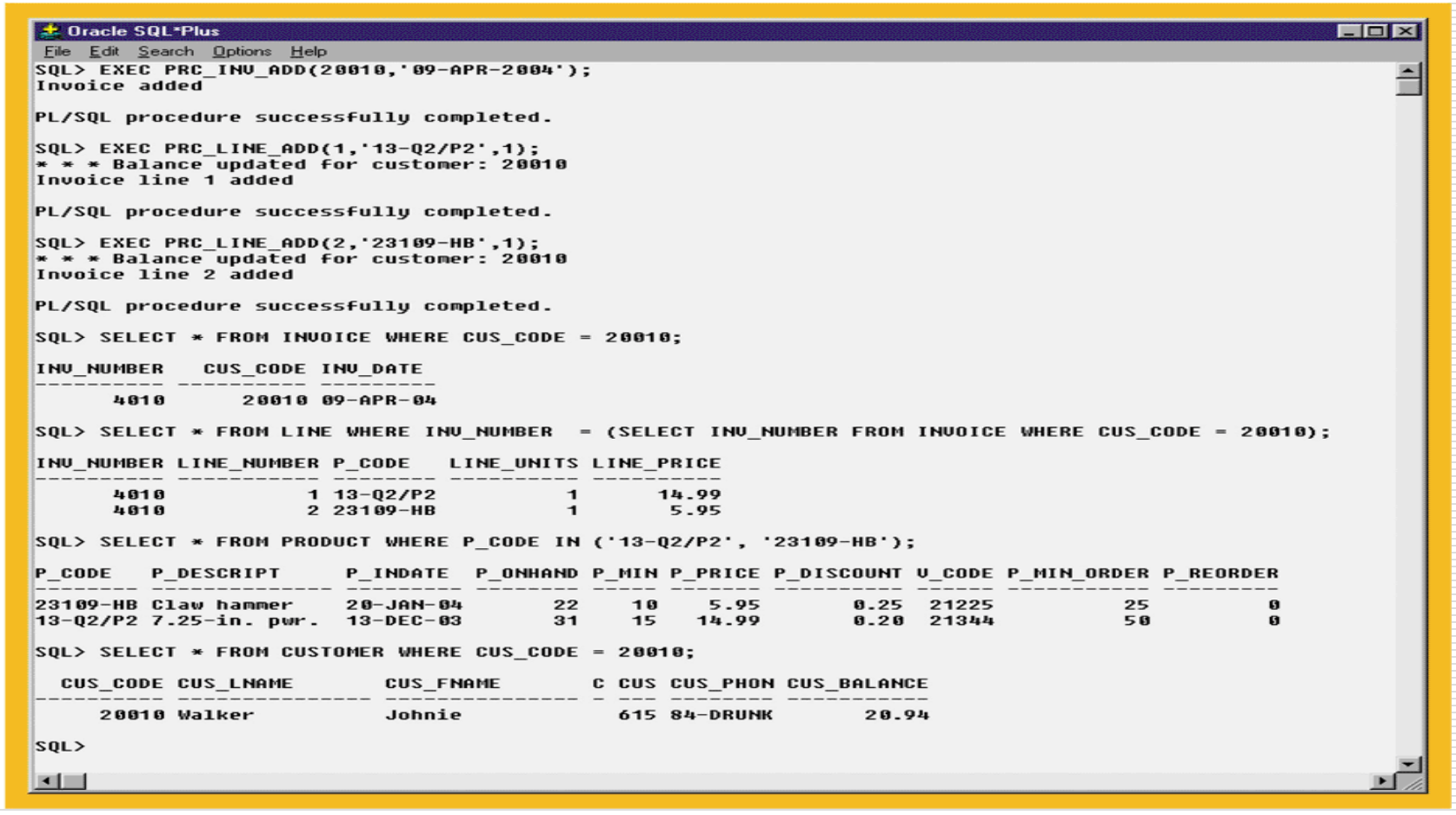
The PRC_INV_ADD and PRC_LINE_ADD Stored Procedures

FIGURE 7.46 THE PRC_INV_ADD AND PRC_LINE_ADD STORED PROCEDURES



Testing the PRC_INV_ADD and PRC_LINE_ADD Procedures

FIGURE 7.47 TESTING THE PRC_INV_ADD AND PRC_LINE_ADD PROCEDURES



Embedded SQL

- SQL statements that are contained within an application programming language
- In some programming languages the SQL code is preceded by EXEC SQL and followed with END-EXEC
  - SQLSTATE and SQLCODE reporting variables inform the program of the status of the SQL code that executed

Dynamic SQL

- SQL statement is not known in advance, but instead is generated at run time
- Program can generate SQL statements at run time that are required to respond to ad hoc queries
- Attribute list and the condition are not known until the end user specifies them
- Tends to be much slower than static SQL
- Requires more computer resources